CONTINUED

Procedural

This has statements that are run in the order that they are written. It makes use of subroutines (procedures and functions) that can be called from other parts of the program. This is the type of language that you are most likely to start learning to program with.



Figure 7.1: A person reading programming statements

Discussion questions

1 What are the key features of an event driven language?

2 Why do you think there is a need for different types of programming language? Can you think of any types of problem that lend themselves to one type of language?

7.1 Program development life cycle

When a person, or organisation, creates a new computer program they will use a structured, organised plan of how to create the program. This is called the **program development life cycle**.

There are several varieties of life cycle, including cyclic, spiral, waterfall and rapid development. These all include the stages **analysis**, design, coding and testing.

KEY WORDS

program development life cycle: a series of structure step / activities that are followed to produce a system.

analysis: the first stage of the program development life cycle that involves investigating the problem.

7.2 Analysis

This first stage involves looking at the problem and the system that is needed. The problem is explored, and the requirements of the program are identified. **Decomposition** is used to split the problem into subproblems. Subproblems allow the developers to identify the requirements for each component part, and then tackle these separately in the next stage. Once you have decomposed the problem, each of the subproblems can be implemented as a subprogram.

7.3 Design

Once the requirements have been identified, the program designers can begin planning how the program will work in the **design** phase. This can include an overview of the program using a structure diagram, and the designing of algorithms using flowcharts and pseudocode.

Structure diagram

A **structure diagram** is developed by decomposing a program into its subprograms. The diagram has the name of the program at the top, and below this its subprograms. Each subprogram can be split down further as well if required.

Example 1

A calculator needs to allow the user to enter numbers and the symbol, calculate the result and output this.

The name of the program is at the top. The boxes below are ready for subprograms (Figure 7.2).

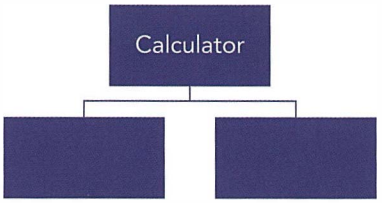


Figure 7.2: A structure diagram for a calculator

One way of splitting up a small program is to think of inputs, processes and outputs. A calculator has an input, processes and output, so these can be added to the diagram (Figure 7.3).

The subprograms can then be split further. The calculator needs two numbers and a symbol to be entered (e.g. 2 + 3). Add these boxes below input (Figure 7.4).

The **process** is dependent on the symbol entered. In this calculator, there is +, − , * and /. Each of these could be given a separate box (Figure 7.5).

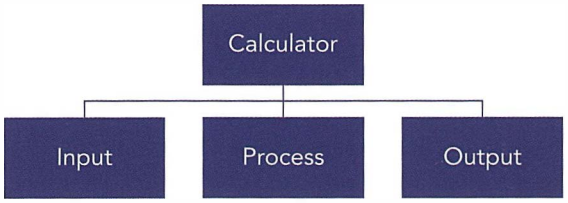


Figure 7.3: Input, process and output are added to the calculator structure diagram

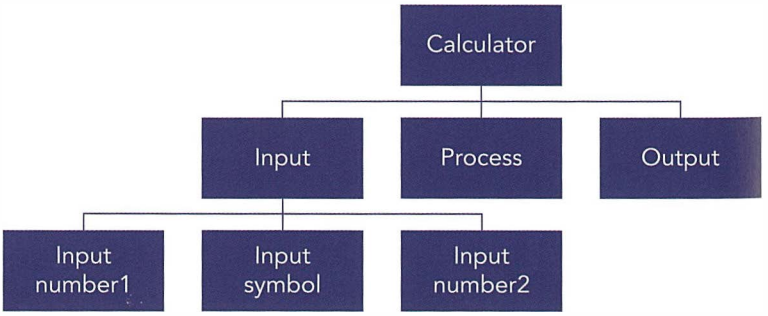


Figure 7.4: Expanding the input subprograms

KEY WORDS

decomposition: taking a system and splitting it into smaller sub-systems, which can in turn be split into smaller sub-systems.

design: the second stage of the program development life cycle, which involves decomposition of the problem and algorithms created ready for implementation.

structure diagram: a hierarchical diagram that shows the decomposition of a system.

process: an action that is performed, for example, `X ← X + 1.`

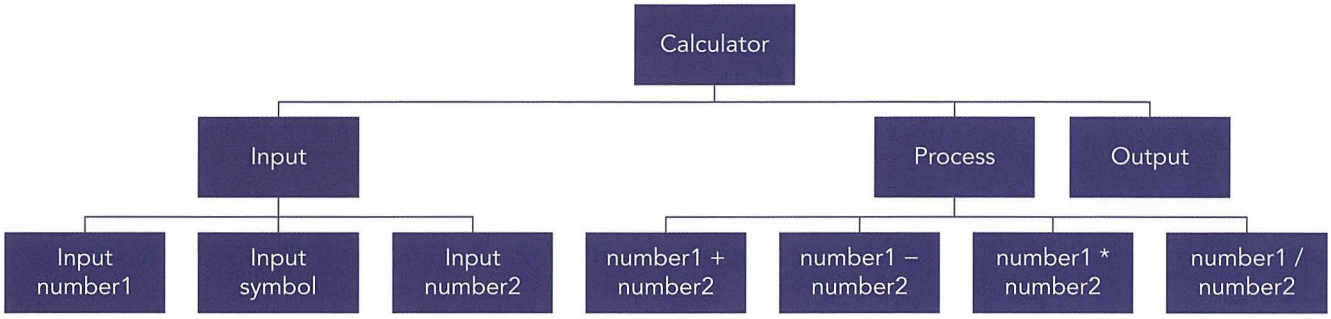


Figure 7.5: Expanding the process subprograms

Or the variable names from the input can be used (Figure 7.6):

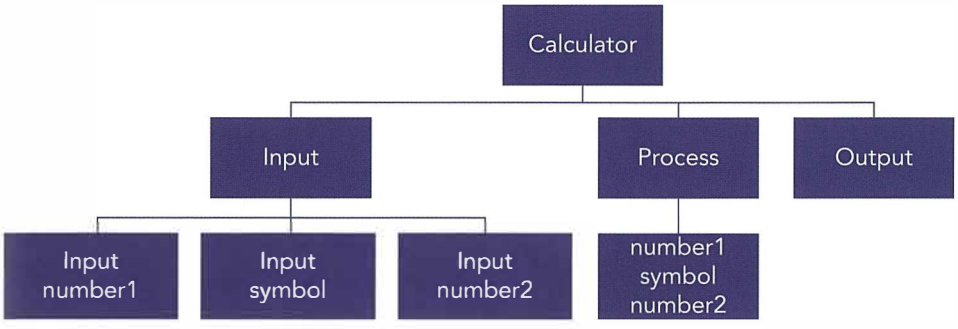


Figure 7.6: An alternative way of representing the algorithm shown in Figure 7.4

Both of these are valid. There are lots of ways you can **decompose** a problem, and everyone could do it slightly differently but the solution is still valid.

Finally, the output needs identifying (Figure 7.7):

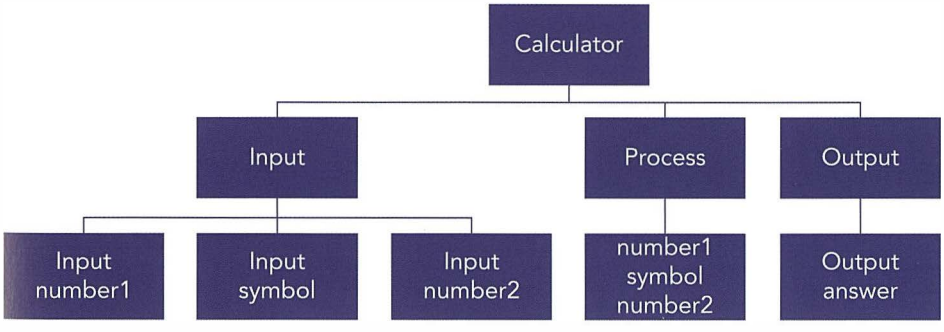


Figure 7.7: A finished structure diagram for the calculator

This program was fairly small, so it has been split into very precise instructions that can then be programmed. The next example is a larger problem.

A computerised version of chess is being developed. Two players can enter their names and then a new board will be displayed. The players take it in turns to select pieces to move and the positions to move them to. The program will check that the moves are valid and then move the pieces. It will remove any pieces that have been taken in each move, and check if anyone has reached checkmate.

KEY WORD

decompose: the action of performing decomposition; splitting a system into smaller sub-systems, which can in turn be split into smaller sub-systems.

Example 2

The name of the program is at the top (Figure 7.8).

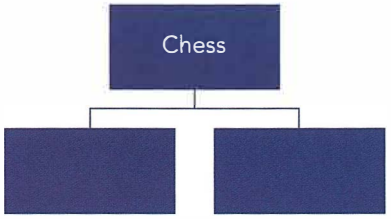


Figure 7.8: A structure diagram for a chess program

The program could be decomposed into inputs, processes and outputs, but this is a large program so it can be split down by different functions instead. The first part is that there are two players who need entering, this is given the name 'New players'. Then the game is set up ('New game'). The players can then move their pieces ('Make move'). The program will check if someone has checkmate ('Check won') (Figure 7.9):

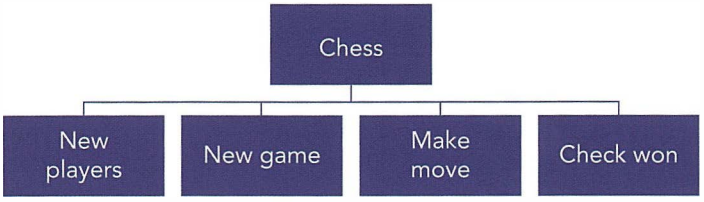


Figure 7.9: Subprograms for the chess program

These can then be split further. They are each given their own sub-diagram here so that they are easier to view (Figures 7.10–13):

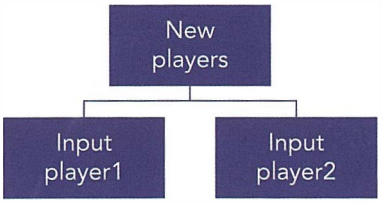


Figure 7.10: The New players subprogram

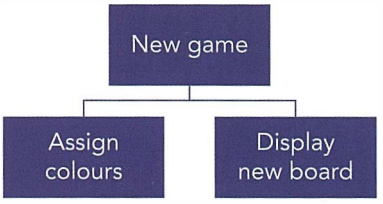


Figure 7.11: The New game subprogram

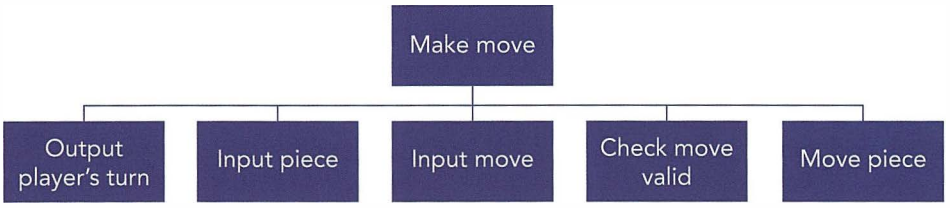


Figure 7.12: The Make move subprogram

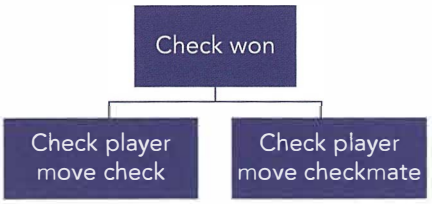


Figure 7.13: The Check won subprogram

ACTIVITY 7.1

Can you split any of these subprograms down further? In 'Check won' (Figure 7.13) what should happen if the result is checkmate? Should there be an output? What if there is check but not checkmate? Should this be output?

Peer Assessment

Compare your subsystems with a friend. Did you have the same answers? If so, are they both valid structure diagrams even though they are different?

Once you have decomposed your program into subprograms, you will need to design an algorithm for each part. This could be the level 1 (the first row below the top box) like in the calculator example. It could be for a lower level, like in the game of chess. Each box should become an individual algorithm. When you write your program, each of these individual algorithms might be a separate function or procedure (see Chapter 8, Section 12). Together, these individual algorithms should come together to create the whole system.

Questions

1. Describe the process of decomposition.
2. A program asks the user to enter two numbers. The program then adds the numbers together and outputs the total. Draw a structure diagram for the program.
3. A satellite navigation system needs to ask the user to input their destination. It then finds the route to the destination and outputs the instructions to the user. Draw a structure diagram for the system.
4. A login system asks the user to enter their username and password. It checks these are valid and either outputs that they are correct, or incorrect and prompts to re-enter. Draw a structure diagram for the system.

Flowcharts

A **flowchart** is a diagrammatic representation of an algorithm. Figure 7.14 shows some of the set of standard symbols.

KEY WORD

flowchart: a diagrammatic representation of an algorithm.

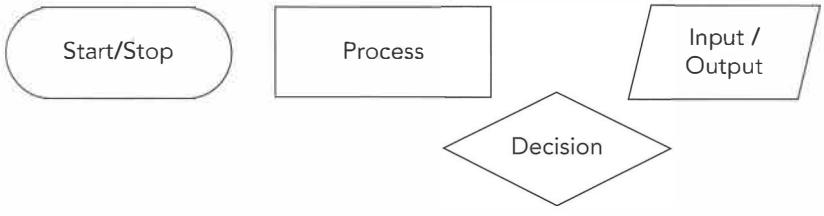
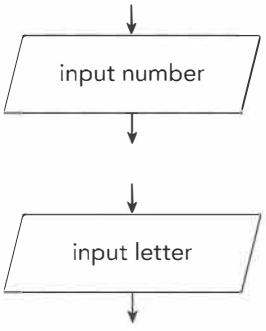
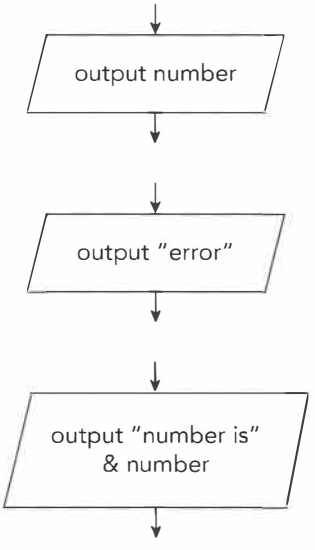


Figure 7.14: Flowchart symbols

A flowchart shows each step of a process, in the order that each step is performed. Each shape has one statement within it, for example, input number, or add 1 to x. The shapes are joined with arrows to show the direction of flow and the content inside each box can be written as words, or as pseudocode statements. Table 7.1 shows examples of how the flowchart symbols are used.

Shape	Description	Examples
Start/stop	A flowchart begins with a Start box, with one arrow from it. A flowchart ends with a Stop box, with one arrow going into it.	 **Figure 7.15:** Start/Stop
Input	A single input of a value with an identifier for that value. One arrow should go into the box, and one arrow should come out.	 **Figure 7.16:** Input
Output	An output of text in speech marks, and/or the identifier of a value	 **Figure 7.17:** Output

(continued)

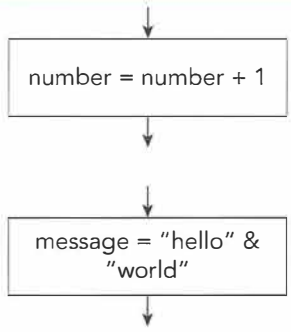
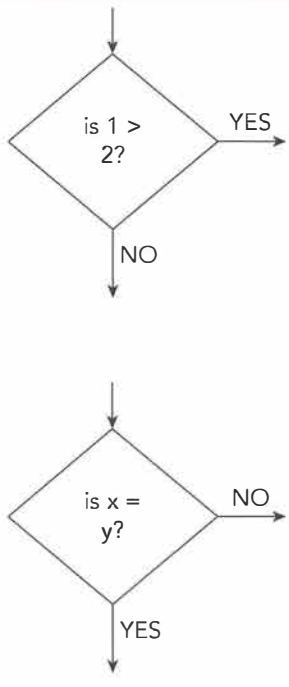
Shape	Description	Examples
Process	An action that is being performed, usually a mathematical calculation. One arrow goes into the box, and one arrow comes out of the box.	 **Figure 7.18:** Process
Decision	A question with two outcomes, yes or no (or true or false). The question is a comparison from: = equal to > greater than < less than >= greater than <= less than. You can have more than one condition by including AND or OR. See Chapter 8, Section 6 for the use of these operators. One arrow goes into the box. Two arrows come out, one labelled yes (or true) and the other no (or false).	 **Figure 7.19:** Decision

Table 7.1: Flowchart symbol examples

It is important that all arrows are complete, i.e. all boxes lead somewhere else so that you cannot get stuck in the flowchart with nowhere to go.

Example 1

Figure 7.20 shows a flowchart for a calculator.

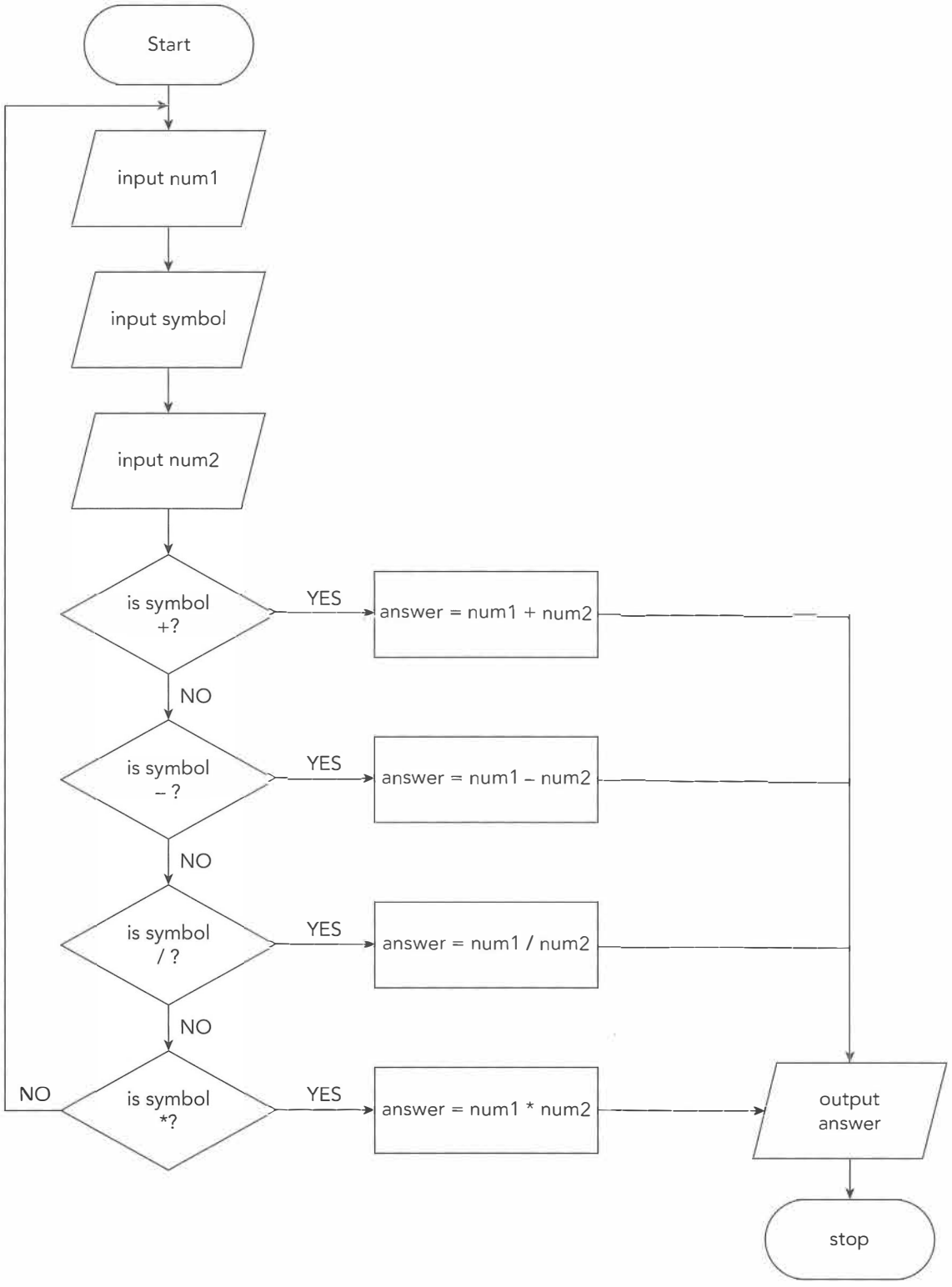


Figure 7.20: A flowchart for a calculator

Example 2

Figure 7.21 shows a flowchart for a password validation program.

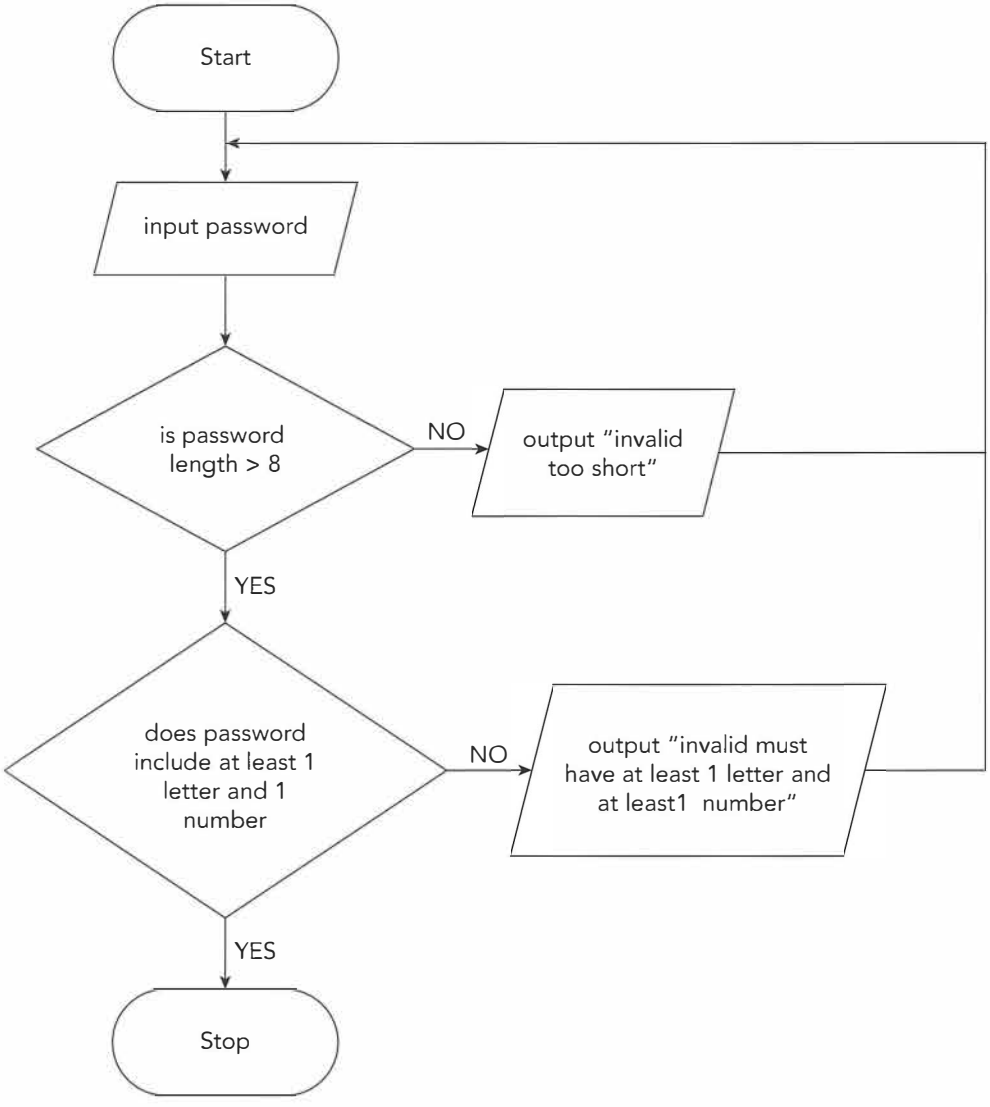


Figure 7.21: A flowchart for a password validation program

ACTIVITY 7.2

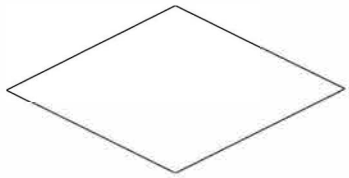
Flowcharts can be used for more than writing programs. Work in pairs to produce a flowchart for making a cup of tea, or coffee, or another drink.

Questions

5 Write the name of each flowchart symbol in the shape.

a 

b 

c 

d 

6 Draw the flowchart symbol for each statement:

a Start

b INPUT x

c INPUT y

d IS x > y?

e x = x + y

f Stop

7 Draw a flowchart to take three numbers as input, find and output the largest number.

7.4 Pseudocode

Pseudocode refers to any code that is not written on a computer to run. There is no one set pseudocode, because if there was then this would be code with a specific language. Instead it's a term for any readable code-like statements that all programmers will understand. This means that it uses keywords, and constructs such as IF statements, WHILE loops, etc. These constructs occur in almost all languages, so any programmer would be expected to read them and translate them into the actual programming language they are using.

KEY WORD

pseudocode: code-like statements that are used to design an algorithm but do not follow any specific language.

Example pseudocode:

```
INPUT Number1
IF Number1 < 10 THEN
    OUTPUT "too small"
ELSE
    OUTPUT "valid"
ENDIF
```

This uses the keywords INPUT, IF and OUTPUT. These can then be translated into whatever language the programmer is using. Here the pseudocode program is converted into Python, VB.NET and Java.

Python

```
number1 = input()
if number < 10:
    print("too small")
else:
    print("valid")
```

VB.NET

```
number1 = console.readline()
if number < 10 then
    console.writeline("too small")
else
    console.writeline("valid")
endif
```

Java

```
number = Integer.parseInt(scanner.nextLine());
if (number < 10){
    System.out.println("too small");
}else{
    System.out.println("valid");
}
```

There are many different valid pseudocode formats. Some will always use capitals for the commands, e.g. INPUT, IF, FOR. Some will use ← instead of an = for assignment. If you write your pseudocode in almost perfect 'code', i.e. you actually write Python code, because it is not to be run and you can have all sorts of syntax errors it is still pseudocode. What is important is that the program is split into steps that can be easily converted into a real program. Table 7.2 shows some examples of valid and invalid pseudocode.

Valid pseudocode	Invalid pseudocode
INPUT Value FOR X = 0 to 9 OUTPUT Value + X NEXT X	Input the value X Loop 10 times In each loop output the loop number added to the input value
num1 = input num2 = input if num1 > num2 then print(num1) else print(num2) Endif	input = num1 input = num2 if num1 is greater than num2 then output num1 if num1 is not greater than num2 then output num2

Table 7.2: Valid/invalid pseudocode examples

TIP

The syllabus includes information about the pseudocode conventions that will appear within examinations. The pseudocode conventions in this book don't always exactly follow the conventions you will see in your exams, but they are still valid. Remember that your syntax doesn't have to be perfect – it is your logic that is important.

PROGRAMMING TASK 7.1

A computer game stores a number for the user to guess. The user has up to 10 attempts to guess the number. After each turn the program outputs whether the number they guessed is smaller, larger, or equal to the number they are trying to guess. If they have 10 guesses without getting it correct, tell them what the answer is. If they get it correct, output the number of attempts it took them.

Getting started

1. Play the game with a partner. Discuss the different steps that you follow. Use this to Identify the inputs, processes and outputs required by this problem.
2. Recap the different components of a structure diagram and the purpose of decomposition.
3. Read an algorithm you have previously written that makes use of: input, output, selection and loops. Revisit the steps you identified in Task 1, which of the four areas is needed in each part.

Practice

1. Decompose the system into its subsystems by creating a structure diagram.
2. Draw a flowchart for the game.
3. Convert your flowchart into pseudocode.

Challenge

1. Is your code efficient? Are there any variables that are used that don't need to be? Can you reduce the number of lines of code?
2. Amend your program so that each time it runs, a random number is generated for the user to guess.

REFLECTION

After completing Programming Task 7.1:

How did you approach the programming task? Did you skip the getting started or did you jump straight to the practice?

Did that approach work? Will you consider the getting started next time?

Did you tackle the challenge yourself?

If not, would you revisit this when you are feeling more confident in this topic?

ACTIVITY 7.3

Find an example of a flowchart and turn it into a pseudocode algorithm.

Questions

8 Describe the purpose of pseudocode.

9 Tick one box in each row to identify if the statements are appropriate pseudocode statements, or not.

Statement	Valid	Invalid
`x = x + 1`		
`INPUT z`		
`IF y < 6 THEN`		
`add 3 to the value entered`		
`store the value entered in the variable newValue`		
`value1 = INPUT("Enter a number")`		
`IF number is more than 10`		
`WHILE number1 <> 0`		

10 Write a pseudocode algorithm to:
- take three values as input
- multiply the numbers together
- output the result.

11 Write a pseudocode algorithm to:
- take 10 numbers as input (using a loop)
- add together the values
- output the total with an appropriate message.

7.5 Coding

Once you have decomposed your problem into subproblems, and you have designed the algorithms using flowcharts and/or pseudocode then you can start writing the program in your chosen programming language. This is often referred to as **coding**.

This will also include some iterative testing. This is testing that is carried out while the program is being developed, for example, if you write the program code for one of the subproblems, then you will test it with a range of different data to make sure it is fully working before moving onto the next step.

7.6 Testing

When you have finished your program, you need to carry out **testing** to make sure it:

- fully works
- does not crash
- meets all requirements.

You will need to identify appropriate **test data** to use to test the program.
There are four types of test data:

- **Normal** – data that the program should accept.
- **Abnormal** – data that the program should not accept.
- **Extreme** – data that is at the edge of what is allowed.
- **Boundary** – data that is on the edge of being accepted and being rejected.

KEY WORDS

coding: the writing of a program using one or more programming languages.

testing: repeated use of a system to try all different possibilities to make sure the system is fully working and cannot be broken.

test data: the input data that is used to test a system.

normal test data: data that a program should accept.

abnormal test data: data that a program should not accept.

extreme test data: data that is on the edge of what is allowed.

boundary test data: data that is on the edge of being accepted, and data that is on the edge of not being accepted.

ACTIVITY 7.10

Trace the algorithm in Skills Focus 7.4 with the following inputs:

a 1, 6, 8, 2, 0

b 1, −1, 2

c 10, 20, 30, −5

Self Assessment

Trace the algorithms a second time and see whether you got the same result. If not, where did you go wrong?

7.9 Finding the purpose of an algorithm

A trace table can help you work out the purpose of an algorithm. By following an algorithm step-by-step you can see what happens to the values, what changes are made, and when these are made. From this you can determine the purpose of the algorithm.

SKILLS FOCUS 7.5

FINDING THE PURPOSE WITH A TRACE TABLE

Describe the processes in this algorithm and state its purpose.

```
INPUT Value1
INPUT Value2
INPUT Value3
IF Value1 > Value2 AND Value1 > Value3 THEN
   OUTPUT(Value1)
ELSEIF Value2 > Value1 AND Value2 > Value3 THEN
   OUTPUT(Value2)
ELSE
   OUTPUT(Value3)
ENDIF
OUTPUT(Value1 + Value2 + Value3)
```

1 First test the algorithm with a set of data that you come up with (if they are not provided for you). For example, test this algorithm using 1, 2 then 3.

2 Create a trace table by following the algorithm with the data input.

Value1	Value2	Value3	OUTPUT
1	2	3	3
			6

3 Repeat the process with a different set of data, for example, this time using 10, 5, 1.

4 Complete the trace table.

CONTINUED

Value1	Value2	Value3	OUTPUT
10	5	1	10
			16

Keep on repeating this process until you can identify the pattern.

In this case it is outputting the largest number input and then outputting the total of the 3 numbers input.

Questions

1 Two sets of data were used to run this algorithm. If you had to choose a third set what would you choose?

2 How do you know when to stop testing the algorithm with different data?

3 Describe the processes in this algorithm and state its purpose using a trace table.

```
Quantity ← 0
Total ← 0
Value ← 0
Small ← 999
WHILE Quantity < 5 DO
    INPUT Value
    IF Value >= 10 AND Value <= 100 THEN
        Quantity ← Quantity + 1
        Total ← Total + Value
        IF Value < Small THEN
            Small ← Value
        ENDIF
    ENDIF
ENDWHILE
OUTPUT("small = ", Small)
OUTPUT(Total / Quantity)
```

quantity	Total	value	small	OUTPUT

TIP

Copy the trace table and test the algorithm in Q3 with some sample data, e.g. 1, 100, 50, 10, 33, 3, 42.

SKILLS FOCUS 7.6

DESCRIBING AN ALGORITHM

When you are describing an algorithm you need to do more than repeat the line of code using English statements. For example, in the following code, for line 02 don't just say, 'it is a for loop from y is 0 to x.' This is stating what each part of the code is, but it does not describe the algorithm.

```
01 INPUT X
02 FOR Y ← 0 TO X
03     OUTPUT Y
04 NEXT Y
```

You can do a trace table to help you work out the purpose of the algorithm before describing it if that helps you. This algorithm has three parts: line 01, line 02 and line 03. These are summarised below:

- Takes a number as input. (line 01)
- Loops from 0 to the number that has been input. (line 02)
- Outputs the values from 0 to the number. (line 03)

From this you can develop a description of the algorithm, for example:

The algorithm takes a number from the user, and then outputs all of the numbers from 0 to the number input.

Questions

1 What methods can you use to work out the purpose of an algorithm?

2 Why should your description explain more than what each part of the code does?

3 Describe the purpose of this algorithm:

```
01 X ← -1
02 Count ← 0
03 WHILE Count < 10 DO
04     INPUT Y
05     IF Y > X THEN
06         X = Y
07     ENDIF
08     Count ← Count + 1
09 ENDWHILE
10 OUTPUT(X)
```

7.10 Finding errors in a program and correcting the errors

To find an error in an algorithm you need to know the purpose of the algorithm. You can then test the algorithm with different data, completing trace tables if needed, to find out what the algorithm actually does. Once you know the difference between what it does and should do, you can start to identify the errors and the changes you need to make.

COMPUTER SCIENCE IN CONTEXT

Computer programmers will come across errors all the time. These could be ones that they have made, or that they are testing or fixing for other people. There are lots of ways that they can find the errors, but many of them will use the two methods used here; using a trace table, and dry running by reading the code. There are other options often available to them as well, for example, features within the Integrated Development Environment (IDE) that they use to write the program. These features allow the programmer to see the values of variables and the outputs at each stage, without having to trace them manually.

SKILLS FOCUS 7.7

USING A TRACE TABLE TO FIND ERRORS IN A PROGRAM

Using a trace table will allow you to follow each step in the algorithm and by comparing this to the intended outcome you can find where it goes wrong, and then work out how to correct it.

For example, this program should take 5 numbers as input and output the total of all the numbers.

```
01 Total ← 1
02 Quantity ← 1
03 WHILE Quantity < 5 DO
04     INPUT Number
05     Total ← Number
06     Quantity ← Quantity + 1
07 ENDWHILE
08 OUTPUT(Total)
```

First dry run the algorithm. If not provided, you will need to identify some test data to use.

This algorithm should take 5 numbers as input, so you could use 1, 2, 3, 4, 5.

Then work out what the result of the algorithm should be. It outputs the total, so it should output: $1+2+3+4+5=15$

CONTINUED

Now draw a trace table for the algorithm (revisit the section on trace tables if needed) and then run each step of the algorithm, writing the values in the table.

Total	Quantity	Number	OUTPUT
1	1		
1	2	1	
2	3	2	
3	4	3	
4	5	4	

Now look at the results and where it has gone wrong:

1. The total is overwritten each time, it should be added to.
2. Total starts with 1 but this will add another 1 to the total, so it should be 0.
3. Only four numbers can be input, it should be five.

Let's take each of the problems one at a time to solve them.

1. The total should not be overwritten. Line 05 overwrites the total. This should be `Total ← Total + Number`.
2. The total should not start at 1, it should be 0. Line 01 writes 1 to total. This line should be `Total ← 0`.
3. Only 4 numbers are input, it should be 5. There are two ways this can be changed, both are valid. Either Line 02 can be `Quantity ← 0` or line 03 can be `WHILE Quantity < 6`.

SKILLS FOCUS 7.8

FINDING ERRORS BY READING THE CODE

You may not always need to use a trace table, or you may not want to. There are many different ways of finding the errors. In this example you will look for each of the requirements in the algorithm to determine whether it has been met.

The following algorithm should take 3 numbers as input, and output the largest value and the average value.

```
01 Total ← 0
02 Largest ← 999
03 FOR X ← 0 TO 2
04     INPUT Data
05     IF Data < 999 THEN
06         Large ← Data
07     ENDIF
08     Total ← Total + Data
09 NEXT X
10 OUTPUT Largest
11 Average ← Total / 3
12 OUTPUT Average
```

CONTINUED

Re-read the description: take 3 numbers as input, and output the largest value and the average value.

This can be split into three sections:

1 Take 3 numbers as input.
2 Output the largest value.
3 Output the average value.

The two outputs need to be calculated before being output, so they can be added to the requirements:

1 Take 3 numbers as input.
2 Calculate the largest value.
3 Output the largest value.
4 Calculate the average value.
5 Output the average value.

Now check the first requirement:

Does it take 3 numbers as input?

The for loop goes from 0 to 2, so it will run 0, 1, 2 = 3 times. Inside the loop it inputs a value each time. **This is correct**.

The second requirement:

Calculate the largest value.

To find the largest it needs to:

a Initialise a largest variable with a low number.

This is **not** met. Line 02 largest is set to 999 but this should be a very small value, e.g.

```
Largest ← -1
```

b It should check if the data input is larger than the data in largest.

This is **not** met. Line 05 checks if data is less than 999. It should be larger.

```
IF Data > Largest THEN
```

c Line 06 replaces the value in `Large` with data, but the identifier is **incorrect**, the variable is named largest.

```
Largest ← Data
```

The third requirement:

Output the largest value.

Line 10 outputs the largest value, **this is correct.**

CONTINUED

The fourth requirement:

Calculate the average value.

The average is calculated by totalling the inputs and then dividing by the number of inputs.

Line 08 totals the data entered, but it uses the **wrong identifier**, it should be data not DATA.

```
Total ← Total + Data
```

Line 11 works out the average **correctly**.

The fifth requirement:

Output the average value.

Line 12 outputs the average **correctly**.

In this case the errors and corrections have been identified along the way. You can also combine this with using test data to check both the original algorithm and to check that your corrections are accurate.

Questions

1 Complete the trace table for the following algorithm with the input data: 20 50 60 25 35.

```
Total ← 0
Average ← 0
Counter ← 0
WHILE Counter <= 5 DO
    INPUT Mark
    Total ← Total + Mark
    Counter ← Counter + 1
ENDWHILE
Average ← Total / Counter
OUTPUT "The total is ", Total
OUTPUT "The average is ", Average
```

Total	Average	Counter	Mark	OUTPUT

CONTINUED

2 **a** Complete the trace table for the following algorithm using the input data: 7.

```
INPUT Number
WHILE Number >= 0 DO
    OUTPUT Number
    Number ← Number - 1
ENDWHILE
```

Number	OUTPUT

b State the purpose of the algorithm.

3 Explain the purpose of the following algorithm:

```
Valid ← 0
Invalid ← 0
Continue ← "Y"
WHILE Continue = "Y" DO
    INPUT Value
    IF Value >= 10 AND Value <= 20 THEN
        Valid ← Valid + 1
    ELSE
        Invalid ← Invalid + 1
    ENDIF
    OUTPUT "Continue?"
    INPUT continue
ENDWHILE
OUTPUT "valid = ", Valid
OUTPUT "invalid = ", Invalid
```